Microsoft 365 Copilot Prompt Pack

Three reusable prompts for project intelligence, TD Cloud operations, and AI/model governance

Included prompts

1	**Project Intelligence and Handover Analyst**	Projects, deadlines, risks, decisions, and five inherited apps
2	**TD Cloud Operations Research and Runbook**	ARE support, pipeline execution, tickets, and CPOps
3	**AI and Model Risk Management (MRM) Guide**	AI lifecycle, development, testing, approvals, and ongoing support

Before you run a prompt: Replace bracketed placeholders with your project names, OneNote links, and application names. Copilot can only inspect sources and applications it can access. Ask it to report inaccessible links and distinguish verified facts from assumptions.

1. Project Intelligence and Handover Analyst

Master prompt for projects and the five inherited applications

Objective

Act as my project intelligence analyst and technical handover assistant. I am an Engineering Manager at TD, and I need a reliable, consolidated view of my projects, deliverables, deadlines, dependencies, risks, decisions, and operational responsibilities.

Investigate the projects I list below using the Microsoft 365 and enterprise knowledge sources available to you. Build a practical reference document that I can use to manage delivery, prepare for meetings, follow up with stakeholders, and take ownership of applications.

Projects to investigate

Project names: [INSERT PROJECT NAMES]

Specific OneNote workbook or notebook links: [PASTE ONENOTE LINKS]

Applications I have inherited:

1. [App name 1]
2. [App name 2]
3. [App name 3]
4. [App name 4]
5. [App name 5]

Additional context or known priorities: [OPTIONAL]

Sources to investigate

Search all relevant sources that you can access, including:

1. **OneNote:** The notebooks, sections, pages, meeting notes, action items, technical documentation, and handover information associated with the named projects and applications. Start with the specific links I provide.
2. **Outlook:** Emails, email threads, attachments, meeting invitations, and follow-up messages that contain decisions, deadlines, commitments, issues, or requests.
3. **Microsoft Teams:** Relevant chats, group conversations, channel discussions, meeting notes, and shared files.
4. **Desktop and work files:** Relevant documents, spreadsheets, presentations, PDFs, text files, and other accessible files.
5. **Browser tabs:** Review currently open tabs where your capabilities permit it. Do not assume that you can inspect browser tabs automatically.
6. **Confluence and linked documentation:** Follow links found in the source material. Review relevant Confluence pages, linked technical documents, architecture diagrams, runbooks, project plans, and procedures if you have access.

Use project names, application names, known acronyms, key stakeholders, related system names, and distinctive terms to find information across sources. Follow relevant links recursively when possible, but prioritize material that directly relates to my projects.

Do not stop after reading the first page or a handful of emails. Investigate relevant historical context, recent updates, and older decisions that still affect current delivery.

Information to extract

1. Project overview

For each project, document:

- Purpose and business problem being addressed.
- Business owner, technical owner, key stakeholders, and support contacts.

- Current status and latest confirmed progress.
- Scope, key deliverables, milestones, and target dates.
- Systems, applications, integrations, data pipelines, and environments involved.
- Dependencies on other teams, vendors, approvals, or technical components.
- Outstanding decisions and unresolved questions.

2. Deadlines and action items

Create a consolidated action tracker with these columns:

- Project
- Action or deliverable
- Owner
- Exact due date
- Status: open, in progress, blocked, completed, or unconfirmed
- Priority: critical, high, medium, or low, with rationale
- Dependency
- Consequence if late
- Direct source link
- Date the information was last confirmed

Extract explicit commitments and deadlines. Also identify implied deadlines, such as an approval required before a scheduled release. Label inferred dates clearly and never present them as confirmed commitments.

Flag overdue actions, deadlines within the next 30 days, missing owners, blocked tasks, and actions that have no confirmed due date.

3. Risks, issues, and dependencies

Identify:

- Technical, operational, security, data, compliance, and delivery risks.
- Production incidents, recurring defects, known limitations, and unresolved support issues.
- Dependencies that could delay a milestone.
- Missing access, approvals, documentation, or technical knowledge.
- Single points of failure and reliance on individual employees.
- Decisions that have not been made but could affect scope or timelines.
- Risks raised in older discussions that may still be unresolved.

For each risk, record the description, impact, likelihood if supported by evidence, mitigation, owner, target resolution date, and source.

Do not assume that an issue is resolved simply because it is no longer mentioned in recent communications.

4. Decisions and project history

Build a decision log containing:

- Decision made or required.
- Date of decision.
- Decision-maker or approving team.
- Reason and relevant alternatives.
- Impact on scope, architecture, budget, timeline, or operations.

- Whether the decision remains current or may have been superseded.
- Source evidence.

Compare conflicting information across emails, Teams, OneNote, and documentation. Prefer explicit, recent decisions from the appropriate owner over informal or older statements. Flag conflicts that cannot be resolved.

5. Technical and operational knowledge

Extract the information I need to understand and support each application or project, including:

- Architecture, environments, integrations, APIs, and data flows.
- Deployment and release procedures.
- Data pipeline execution, monitoring, and troubleshooting.
- Access requirements, permissions, service accounts, and approvals.
- Incident management and escalation procedures.
- Support teams, request processes, ticket queues, and relevant portal links.
- Testing requirements, operational controls, and recovery procedures.
- Known technical debt and planned changes.

Capture exact links, system names, ticket categories, document titles, and procedural steps. Do not invent missing instructions.

6. Stakeholders and communication

Identify who owns key decisions, who performs technical work, who provides approvals, and who should be contacted for support.

Extract important commitments made in conversations and emails. Identify unanswered questions and follow-ups that I need to send.

7. Application handover review

For each of the five inherited applications, use the supplied OneNote workbook as the starting point.

Determine:

- What the application does and who uses it.
- How it is built, hosted, integrated, deployed, and supported.
- Current state and planned changes.
- Recurring operational tasks and known failure scenarios.
- Access, permissions, support contacts, and escalation paths.
- Open tickets, technical debt, outstanding requests, and upcoming deadlines.
- Missing documentation and knowledge that only the previous analyst appears to possess.
- What I should learn or verify during the handover.

Create a separate list of handover questions for the previous analyst or relevant technical owner.

Required output

Organize the final result into these sections:

1. Executive overview and most urgent priorities.
2. Consolidated project portfolio tracker.
3. Master action and deadline register.
4. Key milestones and timeline, ordered chronologically.
5. Risk, issue, and dependency register.
6. Decision log and conflicting information.

7. Individual project summaries.
8. Individual application handover guides.
9. Stakeholder and support contact directory.
10. Missing information, inaccessible sources, and questions to resolve.
11. Recommended next steps for the next 7, 14, and 30 days.

Keep the master trackers consolidated across projects. Avoid creating duplicate actions when the same task appears in multiple sources.

Evidence and accuracy rules

- Attach a direct source link to every important finding whenever possible.
- Include the source date and relevant email subject, OneNote page title, Teams conversation, or Confluence page.
- Distinguish confirmed facts, reported statements, reasonable inferences, and unknowns.
- If two sources disagree, show the discrepancy and explain what needs verification.
- Treat an email or document as historical evidence, not proof that a task is still open.
- Do not mark actions complete without evidence.
- Do not invent dates, owners, project statuses, URLs, procedures, or approvals.
- If you cannot access a link, file, application, or conversation, record it as inaccessible. Do not claim to have reviewed it.
- Respect access controls and do not send messages, create tickets, change files, or modify systems.

Final quality check

Before finishing, verify that you have:

1. Captured all explicitly stated deadlines and commitments found in the accessible sources.
2. Identified the most urgent overdue and upcoming actions.
3. Checked for contradictory or superseded information.
4. Distinguished confirmed deadlines from inferred ones.
5. Included evidence links for key facts.
6. Listed sources that could not be accessed and any important gaps.

The final result should function as a living project reference and handover document, not just a high-level summary. Prioritize actionable details over generic descriptions.

2. TD Cloud Operations Research and Runbook

Operational guide for ARE support, pipelines, tickets, and CPOps

Objective

Act as a technical operations researcher. Search my accessible TD knowledge sources and compile a detailed, evidence-based runbook covering the internal TD Cloud process.

I need to understand how to use TD Cloud, obtain support from AREs, execute and monitor pipelines, submit the correct tickets, and locate the CPOps portal and related resources.

The objective is to create a practical guide that I can follow when performing these tasks, including when I need to troubleshoot an issue or determine the correct team to contact.

Sources to search

Search relevant:

- OneNote notebooks and pages.
- Outlook emails and email threads.
- Microsoft Teams chats, channels, meeting notes, and shared files.
- Accessible desktop files and technical documents.
- Confluence pages and linked internal documentation.
- Existing runbooks, knowledge articles, architecture documents, support procedures, and ticket examples.

Search for variations of the following terms and any internal terminology discovered in the source material:

- TD Cloud, cloud onboarding, cloud access, cloud environments.
- AREs, ARE support, application or engineering support teams, and related internal team names.
- Pipelines, pipeline execution, scheduling, monitoring, failures, reruns, permissions, and troubleshooting.
- CPOps, CPOps portal, internal operations portal, service requests, incidents, and change requests.
- Cloud provisioning, environment setup, access requests, service accounts, networking, approvals, and deployment.
- Tickets, request categories, assignment groups, escalation paths, SLAs, and required forms.

Do not assume what an acronym means. Confirm its meaning from internal documentation.

Research requirements

1. TD Cloud overview

Explain:

- What TD Cloud refers to in the internal context.
- What it is used for and which systems or teams it supports.
- The environments and major components involved.
- Prerequisites for access and operation.
- Required onboarding, training, permissions, approvals, and security controls.
- Which responsibilities belong to my team versus the cloud platform or support teams.

2. ARE support process

Document the exact process for getting help from AREs.

Include:

- What ARE stands for, if confirmed.

- The responsibilities and scope of ARE support.
- When to contact AREs versus another support team.
- How to contact them, including confirmed Teams channels, email addresses, portals, or escalation routes.
- Whether a ticket must be submitted before contacting them.
- Required information, evidence, logs, screenshots, identifiers, and environment details.
- Expected response times or SLAs, if documented.
- Escalation steps when an issue is urgent or remains unresolved.
- Relevant examples from historical requests, with sensitive information excluded.

Provide a numbered procedure that I can follow.

3. Pipeline execution and support

Explain how to run and manage the relevant pipelines.

Find and document:

- Where pipelines are accessed and how to identify the correct pipeline.
- Required permissions and environment prerequisites.
- How to trigger or run a pipeline.
- Required parameters, inputs, configuration, and approvals.
- How to monitor execution and verify successful completion.
- Where logs, execution history, and error details are available.
- Common failures and their documented resolutions.
- How to retry, rerun, recover, or escalate a failed pipeline.
- Whether reruns are safe, whether duplicate processing is possible, and what checks are required.
- Scheduling, dependencies, operational windows, and downstream impacts.
- Who owns the pipeline and who supports the underlying platform.

Separate confirmed step-by-step instructions from details that still need verification. Do not invent commands, portal paths, pipeline names, or execution steps.

4. Ticket submission process

Build a ticketing guide that explains:

- Which portal or system to use for each request type.
- How to distinguish incidents, service requests, access requests, change tickets, and other relevant categories.
- The correct request category, assignment group, and form where documented.
- Required fields and supporting evidence.
- Required approvals and dependencies.
- Priority and severity rules.
- Expected SLAs and escalation routes.
- How to track a request and identify its status.
- What to do when a request is rejected, misrouted, or delayed.

Include a table mapping the issue or request type to the correct ticketing route. Only populate the route when supported by evidence.

5. CPOps portal and internal links

Find the exact CPOps portal link referenced in internal documentation or communications.

Also collect links to:

- TD Cloud documentation and onboarding.
- ARE support resources.
- Pipeline interfaces and runbooks.
- Ticket submission portals and request forms.
- Monitoring and logging tools.
- Access request documentation.
- Relevant Confluence pages and Teams channels.
- Escalation and incident management procedures.

For each link, provide its purpose, when to use it, and the source where it was found.

Do not guess URLs. If you cannot verify the CPOps link, say so and identify the source or team that should confirm it.

6. Operational scenarios

Create concise procedures for these scenarios, if supported by internal evidence:

- I need access to TD Cloud.
- I need help from an ARE.
- I need to run a pipeline for the first time.
- A pipeline fails.
- A pipeline succeeds but the expected data or output is missing.
- I need to rerun a pipeline.
- I need to request a new environment or configuration change.
- I need to submit an access or service request.
- A ticket is blocked or assigned to the wrong team.
- A production issue requires urgent support.

For each scenario, state the first action, required information, correct support route, verification steps, and escalation path.

Required output

Produce a practical runbook with these sections:

1. TD Cloud overview and terminology.
2. Prerequisites and access requirements.
3. ARE support procedure.
4. Pipeline execution and troubleshooting guide.
5. Ticket submission decision table.
6. Verified CPOps link and directory of operational links.
7. Common operational scenarios and procedures.
8. Escalation contacts and support responsibilities.
9. Known failure modes, risks, and operational cautions.
10. Gaps in documentation and questions to ask the relevant teams.

Accuracy requirements

- Search historical communications as well as formal documentation.
- Prefer official, current procedures over informal instructions.
- Check whether older instructions have been superseded.
- Include direct links and source dates for procedural steps.
- Distinguish confirmed processes from suggestions or assumptions.
- Do not infer that a pipeline can be rerun safely without evidence.
- Do not expose credentials, secrets, access tokens, or other sensitive operational information.
- Do not execute pipelines, submit tickets, change configurations, or modify infrastructure.

Finish with a one-page quick reference containing the key portals, first steps, support routes, and escalation procedures. List any critical information that remains unverified.

3. AI and Model Risk Management (MRM) Research and Lifecycle Guide

End-to-end AI governance, model development, testing, and production support

Objective

Act as an internal AI governance and model lifecycle researcher. Search my accessible TD knowledge sources and compile a comprehensive, evidence-based guide to how AI solutions and models must be requested, designed, developed, tested, reviewed, approved, deployed, monitored, and supported within TD.

I need to understand the end-to-end process, including Model Risk Management (MRM), required documentation, stakeholder responsibilities, approval gates, mandatory requests and tickets, testing expectations, production support, and ongoing governance.

The guide must reflect TD's actual internal processes. Do not substitute generic industry practices for internal requirements.

Sources to investigate

Search:

- OneNote notebooks, technical notes, meeting notes, and project documentation.
- Outlook emails, approval threads, requests, and communications with risk, compliance, security, architecture, and AI teams.
- Microsoft Teams conversations, channels, meeting notes, and shared files.
- Accessible desktop files, spreadsheets, presentations, and technical documents.
- Confluence pages, governance standards, policies, internal frameworks, templates, and linked reference material.
- Existing AI and model project documentation, approval records, testing evidence, monitoring procedures, and production support runbooks.

Search for internal terminology and related terms, including:

- AI, generative AI, GenAI, machine learning, ML, large language models, LLMs, and foundation models.
- Model Risk Management, MRM, model inventory, model classification, model validation, model approval, and model monitoring.
- AI intake, use case registration, risk assessments, governance reviews, approvals, and exception requests.
- Model development, experimentation, testing, validation, evaluation, deployment, versioning, and retirement.
- Responsible AI, privacy, data classification, information security, legal review, architecture review, and third-party risk.
- Prompt engineering, retrieval-augmented generation (RAG), agents, AI services, model hosting, and production integrations.
- Model performance, accuracy, bias, explainability, hallucinations, robustness, drift, human oversight, and incident response.
- Ongoing support, monitoring, change management, revalidation, periodic reviews, and audit evidence.

Follow relevant links to internal standards, forms, templates, and Confluence pages wherever access permits.

Research and documentation requirements

1. AI and model governance overview

Explain the internal distinction, if applicable, between:

- Traditional statistical or machine learning models.
- Generative AI solutions and applications.
- Third-party or externally hosted foundation models.
- Internal models, fine-tuned models, and models accessed through APIs.
- AI-enabled features that may not be classified as formal models.

Determine how TD classifies these systems and which governance process applies to each. Do not assume every AI use case follows the same process or that all GenAI solutions are exempt from MRM.

Document the relevant internal teams, their responsibilities, and how to engage them.

2. End-to-end lifecycle

Map the full lifecycle from idea to retirement.

For each stage, document:

- Purpose and entry criteria.
- Required activities and decisions.
- Responsible owner and approving authority.
- Mandatory forms, tickets, requests, assessments, and documentation.
- Required evidence and deliverables.
- Dependencies and approval gates.
- Exit criteria before progressing to the next stage.
- Relevant templates, portals, policies, and source links.

Include the stages supported by internal evidence, such as:

1. Use case identification and initial intake.
2. Business sponsorship and use case approval.
3. AI or model classification and inventory registration.
4. MRM applicability determination and risk tiering.
5. Data, privacy, security, legal, architecture, and third-party assessments.
6. Solution design and model selection.
7. Development and implementation.
8. Testing, evaluation, and independent validation where required.
9. Risk review, remediation, and approval.
10. Production readiness and deployment authorization.
11. Ongoing monitoring, support, and periodic review.
12. Material changes, revalidation, incident handling, and retirement.

Do not present this proposed sequence as TD's official workflow unless the sources confirm it. Identify differences between the documented internal process and this outline.

3. Required requests, tickets, and approvals

Create a consolidated inventory of every relevant request, form, ticket, assessment, and approval discovered.

For each item, capture:

- Exact name.
- Purpose.
- When it is required.
- Who initiates it.
- Responsible team or approver.
- Required inputs and supporting documents.
- Dependencies and approval prerequisites.

- Submission portal or process.
- Expected turnaround time or SLA, if documented.
- Evidence of completion required before proceeding.
- Source link and last confirmed date.

Include relevant MRM, AI intake, model inventory, data access, privacy, information security, architecture, vendor, cloud, deployment, and production support processes where applicable.

Distinguish mandatory requirements from conditional requirements, recommended practices, and items whose applicability remains uncertain.

4. Development requirements

Determine the internal expectations for building AI solutions and models.

Document:

- Approved platforms, services, models, and development environments.
- Data sourcing, access, classification, privacy, and retention requirements.
- Architecture and design documentation.
- Model selection and justification.
- Training, fine-tuning, prompt engineering, RAG, or agent development requirements where relevant.
- Version control, code review, reproducibility, and change tracking.
- Security controls, secrets management, access restrictions, and data leakage prevention.
- Documentation of limitations, assumptions, intended use, and prohibited use.
- Human oversight and fallback procedures.
- Required reviews before testing or production.

Include the exact internal standards and templates when available.

5. Testing, validation, and evaluation

Build a detailed testing checklist based on internal requirements.

Investigate:

- Unit, integration, functional, and end-to-end testing.
- Model performance and task-specific accuracy.
- GenAI evaluation methods, including groundedness, hallucinations, retrieval quality, and response relevance where applicable.
- Bias, fairness, explainability, and robustness requirements.
- Privacy, security, prompt injection, data leakage, and adversarial testing.
- Performance, latency, capacity, and reliability.
- Human review, acceptance criteria, and user acceptance testing.
- Independent model validation and reviewer independence where required.
- Test datasets, evidence retention, defect remediation, and retesting.
- Approval requirements and production release criteria.

For each test, state the applicable scenario, required evidence, responsible party, acceptance criteria, and source.

Do not invent numerical thresholds or mandatory test cases. If internal standards do not specify a threshold, flag it as an open question.

6. MRM-specific process

Investigate the full internal MRM workflow.

Document:

- How to determine whether MRM applies.
- How a model or use case is registered and classified.
- Model risk tiering or materiality assessments.
- Required model documentation and supporting evidence.
- Model development standards and validation expectations.
- Roles of model owners, developers, validators, business owners, and risk teams.
- Review and approval gates.
- Limitations, findings, remediation plans, and exception handling.
- Conditions required before production use.
- Model inventory updates and recordkeeping.
- Periodic review, ongoing monitoring, material change assessments, and revalidation.
- Model retirement and inventory closure.

Capture exact internal terminology, forms, policies, and approval responsibilities.

Where TD uses a separate process for GenAI or non-model AI systems, document the distinction and the criteria for each pathway.

7. Production support and ongoing governance

Explain what happens after deployment.

Cover:

- Operational ownership and support responsibilities.
- Monitoring metrics, evaluation frequency, alerts, and thresholds.
- Performance degradation, model drift, and changing data quality.
- Changes to models, prompts, datasets, retrieval indexes, tools, APIs, or system architecture.
- Determining whether a change requires additional review, approval, or revalidation.
- Incident classification, escalation, remediation, rollback, and fallback.
- User complaints, harmful or incorrect outputs, and control failures.
- Periodic reviews, reporting, audit evidence, and record retention.
- Vendor or model provider changes, version deprecation, and service outages.
- Ongoing access reviews, cost controls, capacity planning, and service reliability where applicable.
- Retirement, replacement, and removal of model access.

Distinguish ongoing operational support from model risk oversight and independent validation.

8. Practical scenario guides

Provide documented procedures for these scenarios:

- I have a new AI use case and need to determine where to start.
- I want to build an internal GenAI application using an existing foundation model.
- I need to access a model or AI service.
- I need to determine whether MRM applies.
- I need to register a model or use case.
- I need to prepare for model validation or a governance review.
- Testing identifies a material issue or unacceptable output.

- I need approval to deploy a solution.
- I need to change a model, prompt, dataset, or integration after launch.
- A production AI solution begins producing incorrect or unsafe outputs.
- I need to complete a periodic review or address an audit finding.
- I need to retire a model or AI solution.

For each scenario, provide the starting point, required request, responsible team, required evidence, approval sequence, and relevant links, but only where supported by internal documentation.

Required output

Produce these deliverables:

1. Executive overview of TD's AI and model governance framework.
2. End-to-end lifecycle diagram or ordered workflow.
3. Request, ticket, assessment, and approval register.
4. MRM applicability and classification guide.
5. Development and architecture checklist.
6. Testing and validation checklist.
7. Production readiness and release checklist.
8. Ongoing monitoring and support runbook.
9. Change management, incident response, and retirement procedures.
10. Directory of internal portals, policies, templates, Confluence pages, and support contacts.
11. RACI-style responsibility matrix for the major lifecycle activities.
12. Open questions, conflicting guidance, and missing documentation.

Evidence and accuracy requirements

- Prioritize current official policies, standards, and approved procedures.
- Use emails and Teams conversations to discover practical implementation details, but verify them against formal guidance where possible.
- Include direct links to source pages, forms, templates, and request portals.
- Record publication or update dates when available.
- Distinguish mandatory controls, conditional requirements, recommendations, and unverified information.
- Flag contradictions between official policy and informal advice.
- Never invent a policy, approval, threshold, form name, SLA, or exemption.
- Do not treat the absence of a search result as proof that a requirement does not exist.
- Respect internal access controls and avoid reproducing credentials, secrets, or unnecessary sensitive data.
- Do not submit requests, modify models, deploy code, or change system configurations.

Finish with a concise quick-start guide: what I need to do first when a new AI or model initiative comes to me, what must be confirmed before development, what must be completed before production, and what must continue after deployment.